

# High-Performance Data-Plane Proxy

C++ and Rust Inline Components, Explained in Simple Terms

---

[Summary](#) · [Architecture Diagram](#) · [Components](#) · [Workflows](#) · [Results](#)

## The project statement being explained

“Increased request-processing throughput 30 to 40% and cut p99 inspection latency 25% by building C++ and Rust inline data-plane components for HTTP parsing, header normalization, rule matching, connection state, and policy execution in an Envoy/Nginx-style event-driven proxy, applying async non-blocking I/O, connection pooling, bounded memory, worker-thread isolation, low-copy hot paths, and CPU, lock-contention, and tail-latency profiling.”

*Note: the design described here is a typical, industry-standard way to build this kind of proxy. Specific numbers (for example 4.0 ms → 3.0 ms) and the per-optimization gains are illustrative examples consistent with the headline results, not measurements from a particular company's system.*

## Contents

*(If page numbers show as blank, right-click the table above in Word and choose “Update Field”.)*

# 1. Overall Summary in Simple Terms

## The one-paragraph version

Every web request that reaches the security platform first passes through a program called a **proxy** – a middle-man that reads the request, checks it against security rules, and passes it on (or blocks it). This proxy runs on every server and handles an enormous number of requests, so every microsecond counts. I rebuilt its most important internal parts – the pieces that read requests, tidy them up, check rules, remember connections and make the final decision – in the fast languages **C++** and **Rust**, and redesigned how they use CPU, memory and network connections. As a result, the same servers now handle **30–40% more requests**, and the slowest 1% of security checks became **25% faster**.

## 1.1 A simple analogy: a busy restaurant kitchen

- **Requests are food orders.** The proxy is the kitchen that must prepare every order quickly and safely.
- **Old way (blocking):** a cook takes one order and stands waiting while the oven heats up, doing nothing else. **New way (async, non-blocking):** the cook puts the dish in the oven, sets a timer, and works on other orders until the timer rings. That is an **event loop**.
- **Worker-thread isolation:** each cook has their own station, knives and ingredients, so cooks never wait for each other to hand over a knife (no **lock contention**).
- **Connection pooling:** instead of calling a supplier and introducing yourself every time, you keep the phone line open and reuse it.
- **Low-copy hot path:** instead of rewriting each order onto a new ticket at every station, everyone reads the same original ticket.
- **Bounded memory:** each station has a fixed-size counter; if it fills up, new orders wait at the door instead of piling up on the floor until the kitchen collapses.
- **Profiling:** a manager with a stopwatch films the kitchen to see exactly where time is wasted, and fixes the slowest spots first.

## 1.2 What each phrase of the statement means

| Phrase                                            | Plain-English meaning                                                                                                                                                                                                  |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Increased request-processing throughput 30 to 40% | Each server can now handle 30–40% more requests per second with the same hardware. That means fewer servers for the same traffic, or more headroom during attacks and spikes.                                          |
| Cut p99 inspection latency 25%                    | “Inspection latency” is how long the security checks take. “p99” is the slowest 1 in 100 requests. Those slowest requests got 25% faster (for example 4.0 ms → 3.0 ms).                                                |
| C++ and Rust                                      | Two programming languages that compile to fast machine code and give precise control over memory. Rust also prevents many memory-safety bugs at compile time – important for code that reads untrusted internet input. |

| Phrase                                          | Plain-English meaning                                                                                                                                               |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Inline data-plane components                    | Code that sits directly in the path of every live request (“inline”), as opposed to background or management code (the “control plane”).                            |
| HTTP parsing                                    | Reading the raw bytes of a web request and understanding its parts: method, URL, headers, body.                                                                     |
| Header normalization                            | Rewriting headers into one standard, clean form (for example lower-case names, decoded values) so rules cannot be fooled by odd spellings.                          |
| Rule matching                                   | Checking the request against thousands of security patterns.                                                                                                        |
| Connection state                                | Remembering what is happening on each open network connection (what has been read, what is pending, timers).                                                        |
| Policy execution                                | Turning rule results into a final action: allow, block, rate-limit, challenge, log.                                                                                 |
| Envoy/Nginx-style event-driven proxy            | The same design used by the famous proxies Envoy and Nginx: a few worker threads, each running an event loop that juggles thousands of connections without waiting. |
| Async non-blocking I/O                          | Never sitting idle waiting for the network; start an operation and get notified when it is ready.                                                                   |
| Connection pooling                              | Reusing already-open connections to backend servers instead of opening a new one per request.                                                                       |
| Bounded memory                                  | Putting strict limits on how much memory any connection, request or worker can use, so the system stays stable under attack.                                        |
| Worker-thread isolation                         | Each CPU core runs its own worker with its own data, so workers do not slow each other down.                                                                        |
| Low-copy hot paths                              | On the most frequently-run code (“hot path”), avoid copying data around in memory.                                                                                  |
| CPU, lock-contention and tail-latency profiling | Measuring precisely where CPU time goes, where threads wait for each other, and what makes the slowest requests slow – then fixing those spots.                     |

### 1.3 The results at a glance

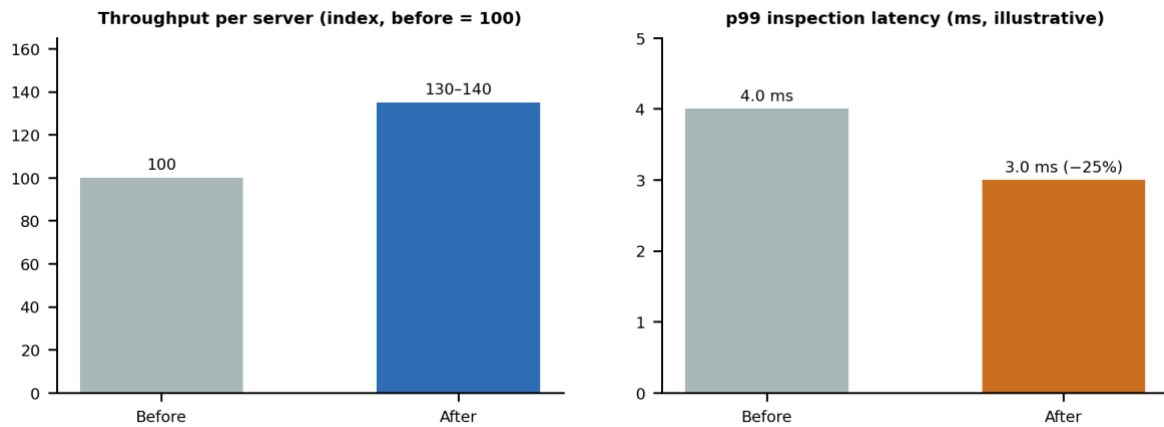


Figure 4 – Before vs. after (throughput is a relative index; latency values are illustrative)

Both results matter for different reasons. **Higher throughput** lowers cost and gives more room to absorb traffic spikes and attacks. **Lower p99 latency** makes the experience consistently fast for everyone – including the unlucky 1% – and keeps the platform inside its overall latency budget.

## 2. Background Concepts in Plain Language

A few ideas make the rest of the document much easier to follow.

| Concept                      | Simple explanation                                                                                                                                                       |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Proxy                        | A program that sits between clients and servers, receiving each request and forwarding it (possibly after checking or changing it).                                      |
| Data plane vs control plane  | Data plane = code handling live traffic (this project). Control plane = code that configures it (settings, rule updates).                                                |
| Thread                       | An independent line of work inside a program. A CPU core runs one thread at a time.                                                                                      |
| Blocking vs non-blocking I/O | Blocking: the thread waits, doing nothing, until data arrives. Non-blocking: the thread asks, and if data is not ready it immediately goes to do other work.             |
| Event loop                   | A loop that repeatedly asks the operating system “which connections are ready?” and handles only those. One thread can juggle tens of thousands of connections this way. |
| epoll / io_uring             | Linux features that tell the program which connections are ready (epoll), or let it submit and complete I/O in batches with fewer system calls (io_uring).               |
| Lock / mutex                 | A “one at a time” sign on shared data. If many threads want it at once they queue up – that waiting is <b>lock contention</b> .                                          |
| Memory copy                  | Moving bytes from one place in memory to another. Each copy costs CPU time and pollutes the CPU cache.                                                                   |
| CPU cache                    | A tiny, very fast memory inside the CPU. Code is fast when the data it needs is already in cache (“cache-warm”).                                                         |
| Allocation                   | Asking for a new piece of memory. Frequent small allocations are surprisingly slow and cause fragmentation.                                                              |
| SIMD                         | CPU instructions that process 16–64 bytes in one step instead of one byte at a time – great for scanning text.                                                           |
| Throughput                   | How much work is done per second (requests per second).                                                                                                                  |
| Latency / p50 / p99 / p999   | How long one request takes. p50 = typical, p99 = slowest 1 in 100, p999 = slowest 1 in 1,000. The slow end is called the <b>tail</b> .                                   |
| Hot path                     | The code that runs for every single request – where optimizations pay off the most.                                                                                      |
| Flame graph                  | A picture of where CPU time is spent: wider bars = more time. It shows at a glance which functions to optimize.                                                          |
| FFI                          | Foreign Function Interface – how code in one language (Rust) calls code in another (C++).                                                                                |

### 2.1 Why the old approach was slower

Before this work, the inspection path had several typical sources of waste. These were found by profiling (Section 5.7) and each was addressed by one or more of the new components:

| Problem found                                                   | Why it hurt                                                             | What replaced it                                        |
|-----------------------------------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------|
| Request data copied into new strings at each stage              | CPU time spent copying; caches flushed                                  | Zero-copy views (slices) into one shared buffer         |
| Many small memory allocations per request                       | Allocator overhead and lock contention inside the allocator             | Per-request arenas and pooled buffers                   |
| Shared global data protected by locks (stats, config, counters) | Workers queued behind each other, especially at high load – hurting p99 | Per-worker data and lock-free read-only snapshots (RCU) |
| Rules checked one by one with separate regular expressions      | Cost grew with the number of rules                                      | One compiled multi-pattern matcher with SIMD            |
| New connection to the backend for many requests                 | Extra network round trips and TLS handshakes                            | Per-worker connection pools with keep-alive             |
| Unbounded buffers under slow or abusive clients                 | Memory spikes, garbage-collection-like pauses, swap, tail latency       | Hard memory caps, watermarks and backpressure           |
| Policy logic interpreted from a generic tree each time          | Many branches and pointer chasing per request                           | Policy compiled into compact bytecode / decision tree   |

### 3. Architecture Diagram

The architecture is shown at two levels: a simple overview (Figure 1) and the detailed internal design (Figure 2). Figure 3 shows the performance-engineering loop that produced the results.

#### 3.1 High-level overview

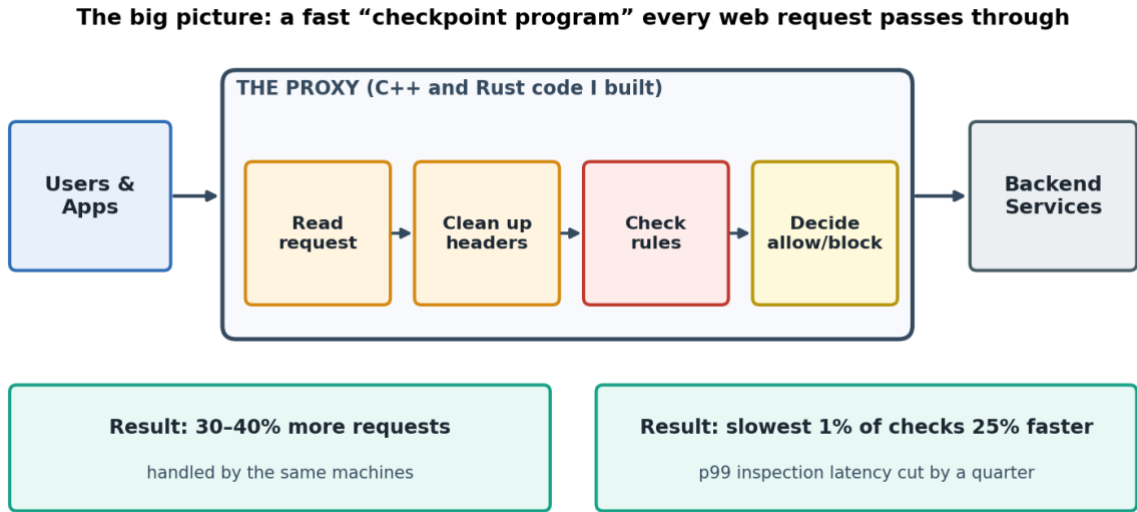


Figure 1 – The big picture: every request passes through the proxy’s four inline stages

**How to read Figure 1:** requests from users and apps (left) enter the proxy. Inside, four stages run in order: read the request (parse), clean up its headers (normalize), check it against the rules (match), and decide what to do (policy). Allowed requests continue to the backend services (right). The two green boxes at the bottom are the outcomes of the project.

## 3.2 Detailed architecture

### Event-Driven Data-Plane Proxy — Detailed Architecture

Numbered circles = order a request follows. Orange = Rust component, blue = C++ component, purple = shared/read-only.

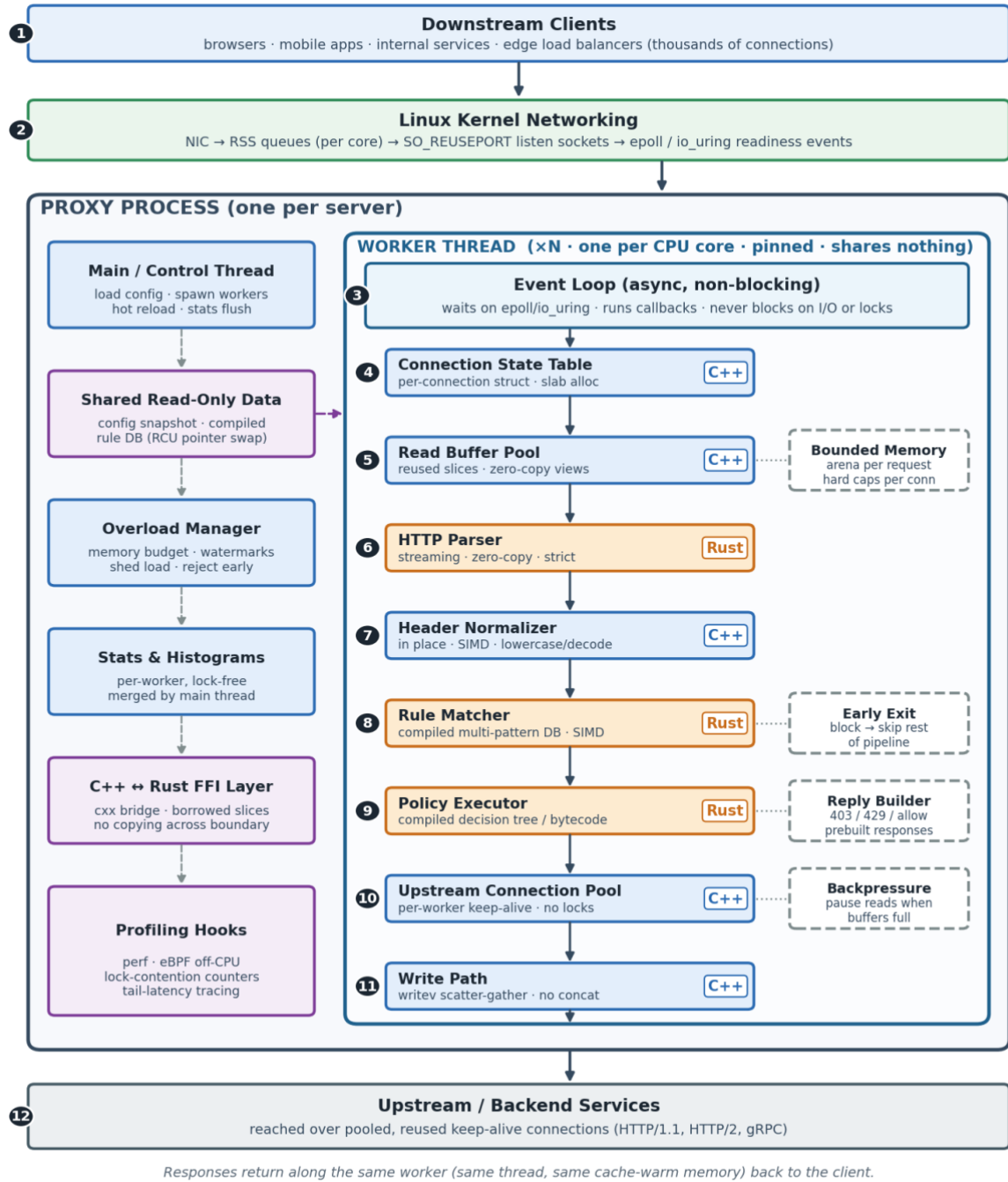


Figure 2 – Detailed internal architecture of the event-driven proxy

### 3.3 Reading the detailed diagram, element by element

#### Visual conventions

| Visual element                      | What it means                                                                                                          |
|-------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Orange box with “Rust” tag          | A component written in Rust (chosen where untrusted input is handled and memory safety matters most).                  |
| Blue box with “C++” tag             | A component written in C++ (chosen where it integrates tightly with the existing proxy core and networking code).      |
| Purple box                          | Shared, read-only or tooling pieces: the config/rule snapshot, the C++↔Rust bridge, and profiling hooks.               |
| Dashed grey box on the right        | A helper behaviour attached to a stage (bounded memory, early exit, reply builder, backpressure). Not a separate step. |
| Dotted grey line                    | Connects a stage to its helper behaviour.                                                                              |
| Solid dark arrow                    | The path a request follows.                                                                                            |
| Dashed grey arrows on the left      | Control relationships managed by the main thread; they are not on the request path.                                    |
| Dashed purple arrow into the worker | Workers read the shared config and compiled rule database without locks.                                               |
| Black numbered circle               | The order of steps a request takes (1 to 12).                                                                          |
| “WORKER THREAD (×N)”                | There is one such worker per CPU core; the diagram shows one of them. All are identical and independent.               |

#### Walking the diagram from top to bottom

| # | Element                 | What happens there (simple terms)                                                                                                                                                                                                                      |
|---|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Downstream Clients      | Anything sending requests to the proxy: browsers, apps, internal services, or an edge load balancer in front.                                                                                                                                          |
| 2 | Linux Kernel Networking | The network card spreads incoming packets across CPU cores (RSS). Each worker has its own listening socket (SO_REUSEPORT), so new connections are shared out without a central bottleneck. epoll / io_uring tells workers which connections have data. |
| 3 | Event Loop              | The heart of each worker. It waits for “ready” events and runs the right small piece of code for each. It never waits on the network or on a lock.                                                                                                     |
| 4 | Connection State Table  | A record for each open connection: its buffers, parser progress, timers and upstream link. Allocated from a fast per-worker slab.                                                                                                                      |
| 5 | Read Buffer Pool        | Incoming bytes land in reusable, fixed-size buffers. Later stages see “views” into these buffers rather than copies. Linked to the Bounded Memory helper.                                                                                              |

| #  | Element                        | What happens there (simple terms)                                                                                                                        |
|----|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 6  | HTTP Parser (Rust)             | Reads the bytes incrementally and identifies method, path, headers and body boundaries, without copying. Rejects malformed or ambiguous requests.        |
| 7  | Header Normalizer (C++)        | Converts header names and values into a canonical form in place, using SIMD to process many bytes at once.                                               |
| 8  | Rule Matcher (Rust)            | Scans the normalized request once against a pre-compiled database of all rule patterns. Linked to Early Exit: a definite block skips remaining work.     |
| 9  | Policy Executor (Rust)         | Runs the compiled policy to decide allow / block / rate-limit / challenge / log. Linked to the Reply Builder for instant pre-built responses.            |
| 10 | Upstream Connection Pool (C++) | Picks an already-open connection to the backend owned by this worker. Linked to Backpressure: if the backend is slow, reading from the client is paused. |
| 11 | Write Path (C++)               | Sends data out using scatter-gather writes, which send several buffer pieces in one system call without joining them first.                              |
| 12 | Upstream / Backend Services    | The real application servers. The response returns through the same worker to the client.                                                                |
| —  | Main / Control Thread          | Loads configuration, starts workers, performs hot reloads, merges statistics. Never handles requests.                                                    |
| —  | Shared Read-Only Data          | The current config and compiled rules. Workers read them freely; updates happen by swapping a pointer (RCU), never by locking.                           |
| —  | Overload Manager               | Watches total memory and CPU; when limits approach, it sheds load early and gracefully.                                                                  |
| —  | Stats & Histograms             | Every worker records its own counters and latency histograms with no locks; the main thread merges them periodically.                                    |
| —  | C++ ↔ Rust FFI Layer           | The bridge that lets C++ and Rust code call each other, passing pointers to the same bytes rather than copies.                                           |
| —  | Profiling Hooks                | Built-in support for CPU profiling, off-CPU (waiting) analysis, lock-contention counters and tail-latency tracing.                                       |

### 3.4 Performance-engineering loop

The performance-engineering loop used for every optimization

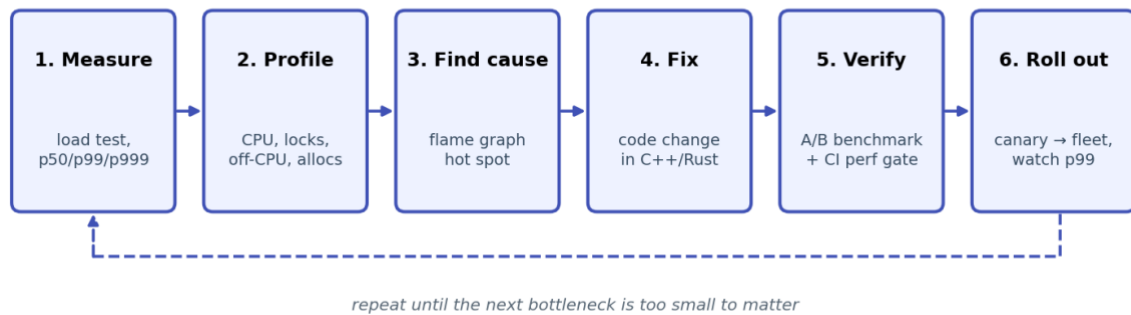


Figure 3 – The measure → profile → fix → verify loop behind every optimization

**How to read Figure 3:** each improvement started with a measurement under realistic load, then profiling to find exactly where time was spent, then a targeted fix, then an A/B benchmark and automated performance test, then a careful rollout. The dashed line shows the loop repeating for the next biggest bottleneck. Section 5.7 walks through it in detail.

## 4. Components Explained in Detail

Every component follows the same layout: a plain-language summary, what it is, why it is needed, how it works step by step, the small details that matter, what happens when things go wrong, and how it contributed to the headline results.

### 4.1 The proxy foundation

#### 4.1.1 Event-driven proxy core (Envoy/Nginx-style)

##### In simple terms

The overall shape of the program: a small number of worker threads, one per CPU core, each juggling thousands of connections by reacting to events instead of waiting.

**What it is:** A multi-threaded, event-driven server process. One main thread manages configuration; N worker threads (N = number of CPU cores) handle all traffic. Each worker runs its own event loop.

**Why it is needed:** The older “one thread per connection” model wastes memory (each thread needs its own stack) and CPU (the operating system constantly switches between thousands of threads). The event-driven model keeps each core busy with useful work.

##### How it works, step by step:

1. At start-up, the main thread reads configuration and compiles rules.
2. It creates one worker thread per CPU core and pins each worker to its own core.
3. Each worker opens its own listening socket and registers it with its own event loop.
4. Each worker then runs forever: wait for events → handle each ready connection a little → go back to waiting.
5. The main thread stays in the background handling reloads, statistics and health checks.

##### Minute details that matter:

- Pinning workers to cores keeps their data in that core’s cache, which is faster.
- Workers are designed to share nothing on the hot path; this is what makes performance scale almost linearly with core count.
- Filter-chain design (like Envoy filters or Nginx modules): each stage is a pluggable filter with a clear interface, so the new C++ and Rust components slot into the existing proxy.

**If it breaks:** If a worker hits a fatal problem, the process supervisor restarts it; because workers are independent, other workers continue serving.

##### Contribution to the results

The foundation that makes worker-thread isolation and async I/O possible. The reduction in thread switching and shared-state contention accounted for a significant part of the throughput gain.

### 4.1.2 Kernel interface: SO\_REUSEPORT, RSS, epoll and io\_uring

#### In simple terms

How the proxy gets new connections and data from the operating system, shared out fairly across CPU cores without a traffic jam at the entrance.

**What it is:** Use of Linux features: **RSS** (the network card spreads packets across per-core receive queues), **SO\_REUSEPORT** (every worker has its own listening socket on the same port), and **epoll** or **io\_uring** (efficient event notification and batched I/O).

**Why it is needed:** With a single shared listening socket, all workers compete for new connections (a “thundering herd”) and the kernel uses locks internally. Separate sockets per worker remove that contention and spread load evenly.

#### How it works, step by step:

1. The network card hashes each connection to one receive queue, each linked to a CPU core.
2. The kernel places the new connection on one worker’s listening socket (SO\_REUSEPORT).
3. That worker’s event loop is told “new connection ready” via epoll / io\_uring.
4. The worker accepts it, sets it to non-blocking mode, and registers it for read events.
5. From then on, the worker is notified only when this connection has data to read or room to write.

#### Minute details that matter:

- Edge-triggered epoll means the worker is notified once per change, then reads until the socket is empty – fewer wake-ups.
- io\_uring lets the worker submit many reads/writes in one batch and collect results in another, cutting system-call overhead.
- Socket options such as TCP\_NODELAY (no artificial send delay) and tuned buffer sizes are set per connection.

#### Contribution to the results

Fewer system calls per request and no contention on accepting connections – a direct contributor to throughput.

## 4.2 Inline data-plane components (built in C++ and Rust)

### 4.2.1 Connection state manager (C++)

#### In simple terms

A tidy record book for every open connection – what has been read, where the parser is, which timers are running, and which backend connection is linked to it.

**What it is:** A per-worker table of connection objects allocated from a slab (a pre-allocated block of same-sized slots), with an efficient timer structure for time-outs.

**Why it is needed:** A worker may handle tens of thousands of connections. Creating and destroying these records with general-purpose memory allocation, or storing them in a shared structure, costs time and causes contention.

**How it works, step by step:**

1. When a connection is accepted, a slot is taken from the worker's slab – no general allocation.
2. The slot stores: socket handle, state (reading headers / reading body / waiting for upstream / writing), buffer references, parser state, timers, counters.
3. Timers (idle, header-read, request-total) are stored in a **timer wheel** – adding, cancelling and firing timers are constant-time operations.
4. Each event updates the state machine: e.g. "headers complete" → run inspection → "waiting for upstream".
5. When the connection closes, buffers are returned to the pool and the slot is returned to the slab.

**Minute details that matter:**

- Fields that are used together are placed next to each other in memory (cache-line friendly layout) and rarely-used fields are separated.
- No locks: only the owning worker ever touches its connections.
- HTTP/2 connections carry many streams; each stream gets a small sub-record in the same slab scheme.

**If it breaks:** Slow or idle connections are closed by timers so they cannot hold resources forever (defends against slow-loris style attacks).

#### Contribution to the results

Removed allocation and lookup overhead per connection and per request; improved cache behaviour.

## 4.2.2 Buffer management and low-copy hot path (C++)

### In simple terms

Instead of making copies of the request at every step, the request is read into one place in memory, and every stage looks at that same place.

**What it is:** A pool of reusable fixed-size buffers plus "slice" (view) objects – small descriptors that say "bytes 120 to 180 of buffer #7" – passed between stages instead of copied strings.

**Why it is needed:** Copying is one of the most common hidden costs in network software. A request that is copied five times costs roughly five times the memory bandwidth, and each copy pushes useful data out of the CPU cache.

**How it works, step by step:**

1. The socket read places bytes directly into a buffer taken from the worker’s pool.
2. The parser returns slices pointing into that buffer for the method, path, and each header name and value.
3. The normalizer changes bytes in place when the canonical form is the same length or shorter; only if it must grow does it write into a small scratch area.
4. The rule matcher and policy executor read the slices directly.
5. On forwarding, the write path sends the original buffer pieces plus any new header bytes together, without joining them (scatter-gather).

**Minute details that matter:**

- Reference counting ensures a buffer is only returned to the pool once no stage still needs it.
- Buffers are sized to fit typical requests (e.g. 16 KB) with chaining for larger ones.
- Across the C++/Rust boundary, slices are passed as borrowed pointers + lengths, so language boundaries do not force copies either.

**Contribution to the results**

One of the biggest single contributors to both results: less CPU per request (throughput) and fewer cache misses and allocator pauses (p99).

**4.2.3 HTTP parser (Rust)****In simple terms**

Reads the raw bytes of a web request and works out its parts – quickly, safely, and without making copies – even if the request arrives in small pieces.

**What it is:** A streaming (incremental) HTTP/1.1 parser written in Rust, integrated with the proxy’s HTTP/2 and HTTP/3 codecs, that produces zero-copy slices.

**Why it is needed:** Parsing untrusted internet input is the most security-sensitive code in the proxy: bugs here cause crashes or request-smuggling vulnerabilities. Rust gives memory safety without slowing things down, and a purpose-built parser avoids the overhead of a general-purpose one.

**How it works, step by step:**

1. Receive whatever bytes are available – maybe only half a header line.
2. Scan forward using SIMD to find delimiters (spaces, “\r\n”, colons) many bytes at a time.
3. Record the positions of the method, target, version and each header as slices.
4. If input ends mid-token, save a tiny state and resume exactly there when more bytes arrive – no re-scanning from the beginning.
5. Validate strictly: legal characters, one Content-Length or valid chunked encoding, no conflicting framing headers, limits on sizes and counts.
6. Emit “headers complete” to the next stage; stream the body in chunks as it arrives.

**Minute details that matter:**

- Strictness prevents request smuggling (when the proxy and backend disagree on where a request ends).
- Limits: maximum request-line length, header count, header size, and total header bytes – exceeding any returns a 4xx immediately.
- Heavily fuzz-tested (random and malformed inputs) and property-tested against reference parsers.
- Resumable parsing avoids the classic performance trap where slow clients cause the same bytes to be re-parsed many times.

**If it breaks:** Malformed input produces a clean 400 response and connection close – never a crash.

**Contribution to the results**

Faster parsing and no copies lowered CPU per request; resumable parsing removed a source of tail-latency spikes with slow clients.

**4.2.4 Header normalizer (C++)****In simple terms**

Rewrites headers into one standard form so that security rules see the “true” meaning, no matter how an attacker spells it.

**What it is:** An in-place canonicalization stage for header names and values, and the URL path and query, built with SIMD routines.

**Why it is needed:** Attackers disguise payloads using mixed case, percent-encoding, double encoding, extra spaces, duplicate headers or path tricks like “/a/./b”. Rules must see a single canonical version. It also needs to be very fast, because it touches every byte of every header.

**How it works, step by step:**

1. Lower-case header names using SIMD (16–32 bytes per instruction).
2. Trim surrounding whitespace and collapse obsolete line-folding.
3. Percent-decode the path and query in place, handling double encoding safely and rejecting invalid sequences.
4. Resolve path segments (“.” and “..”), merge duplicate slashes, and normalize Unicode where configured.
5. Merge or flag duplicate headers according to policy (e.g., multiple Host headers are rejected).
6. Keep both original and normalized slices so rules and logs can use either.

**Minute details that matter:**

- Common header names (Host, Content-Type, User-Agent, etc.) are mapped to small integer IDs, so later stages compare numbers instead of strings.
- Work is done only once per request; later stages never re-normalize.

**Contribution to the results**

SIMD and in-place processing made normalization several times cheaper; header IDs sped up every later lookup.

**4.2.5 Rule matcher (Rust)****In simple terms**

Checks the request against thousands of security patterns in a single fast pass, instead of checking each pattern one at a time.

**What it is:** A matching engine that uses a pre-compiled multi-pattern database (Hyperscan / Vectorscan, or an Aho-Corasick automaton with SIMD prefilters) plus linear-time regular expressions for confirmation.

**Why it is needed:** Checking rules one by one makes cost grow with the number of rules, and some regular expressions can take extremely long on crafted inputs (ReDoS). Both hurt throughput and, especially, p99.

**How it works, step by step:**

1. At config-load time (not per request), all rule patterns are compiled into one database, grouped by which request field they apply to (path, query, specific headers, body).
2. Per request, each field is scanned once against its database. The scan reports every rule whose pattern appeared.
3. Only those candidate rules are then fully evaluated (extra conditions, confirmation regex with a linear-time engine).
4. Scores and matched rule IDs are collected into a small fixed-size result structure.
5. If a rule marked “terminal block” matches, **early exit** skips the remaining fields and goes straight to policy.

**Minute details that matter:**

- The compiled database is immutable and shared read-only by all workers; each worker has its own small scratch space, so there are no locks.
- Per-request time budgets: if a scan would exceed its budget (e.g. a huge body), it stops and the configured fail-open / fail-closed policy applies.
- Rust’s safety guarantees protect this complex code from memory bugs while keeping C-like speed.

**If it breaks:** A rule set that fails to compile or exceeds performance limits is rejected at load time; workers keep running on the previous rules.

**Contribution to the results**

The largest single contributor to the 25% p99 reduction, because rule evaluation was the biggest share of inspection time and the biggest source of slow outliers.

#### 4.2.6 Policy executor (Rust)

##### In simple terms

The judge that takes all the check results and quickly decides what to do: allow, block, slow down, challenge, or just record.

**What it is:** An engine that runs customer and platform policies which have been compiled into a compact decision tree or bytecode program.

**Why it is needed:** Policies can be complex (“if path starts with /api and score > 5 and country not in allow-list, then rate-limit”). Interpreting a generic, pointer-heavy rule tree on every request is slow and unpredictable.

##### How it works, step by step:

1. At load time, each policy is compiled: conditions are ordered (cheapest and most selective first), constants pre-computed, and the result stored as compact bytecode.
2. Per request, the executor runs the bytecode using the facts already gathered (header IDs, match results, client info).
3. It outputs one action plus reasons.
4. The **reply builder** returns pre-built responses (403, 429, challenge) instantly for non-allow actions.
5. For “allow”, the request moves on to the upstream stage, with any header additions (e.g., a request ID) recorded as extra slices.

##### Minute details that matter:

- Bounded execution: bytecode has no unbounded loops, so every policy finishes in predictable time.
- Deterministic and explainable: the same inputs always give the same decision, and reasons are logged.

##### Contribution to the results

Reduced branch mispredictions and pointer chasing; made policy time predictable, which tightened the tail.

#### 4.2.7 Upstream connection pool (C++)

##### In simple terms

Keeps a set of already-open connections to the backend servers so each request can reuse one instead of opening a new one.

**What it is:** A per-worker pool of persistent (keep-alive) HTTP/1.1, HTTP/2 or gRPC connections to each backend cluster, with health tracking, limits and time-outs.

**Why it is needed:** Opening a new TCP + TLS connection costs several network round trips and significant CPU. A shared pool across workers would need locks.

**How it works, step by step:**

1. When a request is allowed, the worker looks in its own pool for an idle connection to the chosen backend.
2. If one exists, the request is sent immediately. For HTTP/2, many requests share one connection as separate streams.
3. If none exists and the pool is below its limit, a new connection is opened asynchronously; the request waits in a small per-pool queue.
4. Responses are read by the same event loop and streamed back to the client.
5. Idle connections are kept for a configured time and closed if the backend misbehaves (circuit breaker).

**Minute details that matter:**

- Per-worker pools mean no locks and cache-warm connection state.
- Limits on max connections, max pending requests and max requests per connection protect backends.
- Load balancing (e.g., least-requests or power-of-two-choices) picks the backend host.

**If it breaks:** Failed connections are retried on another host for safe (idempotent) requests; persistent failures open the circuit breaker so traffic stops going to a broken host.

**Contribution to the results**

Cut connection-setup overhead and the latency spikes it causes; contributed to both throughput and tail latency.

#### 4.2.8 Write path (C++)

**In simple terms**

Sends the (possibly modified) request or response out using as few operations as possible, without first gluing pieces together.

**What it is:** Output code using **writev** / **io\_uring** scatter-gather writes that send a list of buffer pieces in one system call.

**Why it is needed:** Joining original bytes and added headers into a new buffer before sending means another copy and another allocation.

**How it works, step by step:**

1. Build a small list: [original request line] [new/changed headers] [original remaining headers] [body buffers].
2. Submit the list in one write.
3. If the socket cannot accept everything yet, remember the position and wait for a “writable” event.

4. Return buffers to the pool once fully sent.

#### Contribution to the results

Removed the final copy on the hot path and reduced system calls.

## 4.3 Stability and isolation mechanisms

### 4.3.1 Bounded memory and backpressure

#### In simple terms

Every connection, request and worker has a strict memory allowance. If data arrives faster than it can be handled, the proxy politely stops reading until there is room.

**What it is:** A set of limits and mechanisms: per-request memory arenas, per-connection buffer caps, high/low watermarks, global memory budget, and flow control (backpressure) between client and backend.

**Why it is needed:** Without limits, a slow backend or a malicious client sending a huge body can make memory grow without bound. That leads to allocator slow-downs, swapping, or the process being killed – all of which destroy tail latency and availability.

#### How it works, step by step:

1. Each request gets an **arena**: a block of memory from which all its small allocations are taken; at the end, the whole arena is released at once.
2. Each connection's buffered bytes are counted. When they pass the **high watermark**, the worker stops reading from that connection.
3. When the buffer drains below the **low watermark**, reading resumes.
4. The same happens in the other direction: if the backend is slow to accept data, the client side is paused (backpressure), and vice versa.
5. A per-worker and global memory budget is tracked; if it nears the limit, the overload manager acts (Section 4.3.3).

#### Minute details that matter:

- Arenas turn thousands of small malloc/free calls into a couple of bulk operations and avoid fragmentation.
- Limits are configurable per route, e.g. larger bodies for an upload endpoint.

#### Contribution to the results

Removed memory-related latency spikes and made performance stable under attack – important for the p99 improvement.

### 4.3.2 Worker-thread isolation (shared-nothing design)

#### In simple terms

Each worker thread has its own everything – connections, buffers, pools, counters – so workers never have to wait for each other.

**What it is:** An architecture rule: no mutable data is shared between workers on the hot path. Shared data is read-only and swapped atomically.

**Why it is needed:** Locks cause threads to queue; atomic counters shared across cores cause “cache-line bouncing” where the same memory is shuffled between cores. Both get worse as core counts and load grow, and both show up as p99 spikes.

#### How it works, step by step:

1. Per-worker: connection table, buffer pool, arena allocator, upstream pools, stats, rate-limit caches, scratch space for matching.
2. Shared but read-only: configuration and compiled rule database, accessed through an atomically-swapped pointer (RCU – read-copy-update).
3. Anything that must be global (e.g., overall stats) is aggregated periodically by the main thread, not updated per request.
4. Cross-worker messages (rare) go through lock-free queues, handled at the worker’s convenience.

#### Minute details that matter:

- Profiling confirmed the removal of hot locks: lock-wait time on the request path dropped to near zero.
- Data structures are padded to avoid “false sharing” (two workers’ data accidentally sitting in the same cache line).

#### Contribution to the results

Scaling across cores became nearly linear; removal of lock contention was a major factor in both throughput and the p99 reduction.

### 4.3.3 Overload manager

#### In simple terms

A safety valve: when the server is close to its limits, it starts turning away a small amount of work early and cheaply, so everything else stays fast.

**What it is:** A monitor of memory, CPU and queue depth with graduated actions.

**Why it is needed:** A system pushed past its capacity does not slow down gracefully – latency explodes for everyone. Shedding a little load early protects the rest.

**How it works, step by step:**

1. Continuously sample memory use, event-loop lag and pending-work queues.
2. At level 1: shrink idle connection timeouts and stop accepting new keep-alive connections.
3. At level 2: reject new connections or return fast 503 responses to low-priority traffic.
4. At level 3: stop reading new requests until pressure drops.
5. Recover gradually as pressure falls.

**Contribution to the results**

Kept p99 stable during spikes instead of letting queues build.

**4.4 Control and cross-cutting pieces****4.4.1 Main thread and lock-free hot reload (RCU)****In simple terms**

When rules or settings change, the new version is prepared in the background and switched in instantly, without pausing traffic or taking a lock.

**What it is:** The main/control thread plus a read-copy-update mechanism for configuration and compiled rules.

**Why it is needed:** Rule updates are frequent. Pausing workers or locking the config on every read would hurt throughput and create latency spikes.

**How it works, step by step:**

1. The main thread receives a new configuration.
2. It validates it and compiles rules and policies into new immutable objects.
3. It atomically swaps a global pointer to the new version.
4. Each worker picks up the new pointer at the start of its next request; requests already in progress finish with the old version.
5. Once all workers have moved on, the old version is freed.

**Minute details that matter:**

- Readers never lock – they just read a pointer.
- If compilation fails, the swap never happens and the old config stays active.

**Contribution to the results**

Eliminated a lock that every request previously took to read config.

#### 4.4.2 C++ ↔ Rust interoperability layer (FFI)

##### In simple terms

The bridge that lets the Rust parts and the C++ parts work together as one program, passing information without copying it.

**What it is:** A typed bridge (for example using the `cxx` crate or a hand-written C ABI) between the C++ proxy core and the Rust components.

**Why it is needed:** The proxy core is C++; new safety-critical components are Rust. A careless bridge could add copies, allocations or unsafe behaviour.

##### How it works, step by step:

1. Define a small, stable interface: functions that take pointers + lengths (slices) and fixed-size result structs.
2. C++ calls into Rust with borrowed views of its buffers; Rust never takes ownership of C++ memory.
3. Rust returns results in caller-provided structures, so no memory is allocated across the boundary.
4. Errors are returned as codes, never as exceptions or panics crossing the boundary.

##### Minute details that matter:

- Calls are coarse-grained (e.g., “parse this chunk”, “match this request”) so bridge overhead is negligible compared with the work done.
- Tests run under sanitizers (AddressSanitizer, ThreadSanitizer) and Miri for the Rust side.

#### 4.4.3 Per-worker stats and latency histograms

##### In simple terms

Counters and stopwatches built into every stage, recording how many requests passed and how long each stage took – without slowing anything down.

**What it is:** Per-worker, lock-free counters and HDR-style latency histograms for each stage (parse, normalize, match, policy, upstream), merged by the main thread.

**Why it is needed:** p99 cannot be improved – or defended – without accurate measurement of each stage.

##### How it works, step by step:

1. Each stage reads a cheap CPU timestamp at start and end.
2. The duration is added to that worker’s histogram (a simple array increment).
3. Every few seconds, the main thread reads all workers’ histograms and publishes merged p50/p99/p999.

##### Minute details that matter:

- Recording is designed to cost nanoseconds; the stats path was itself profiled.

#### 4.4.4 Profiling toolchain: CPU, lock contention and tail latency

##### In simple terms

The set of “X-ray machines” used to see exactly where time goes – on the CPU, waiting for locks, and in the slowest requests.

**What it is:** Tools and built-in hooks: **perf** and flame graphs for CPU time; **eBPF**-based off-CPU analysis for time spent waiting; lock-contention profiling (e.g., perf lock, mutex wait counters); allocation profiling (e.g., heaptrack, jemalloc stats); hardware counters for cache misses and branch mispredictions; and tail-latency tracing that captures details of only the slowest requests.

**Why it is needed:** Guessing about performance is usually wrong. Averages hide the slow requests that define p99. Each tool reveals a different kind of waste.

##### How it works, step by step:

1. **CPU profiling:** sample the call stack thousands of times a second under load; draw a flame graph; wide bars show where CPU goes.
2. **Off-CPU profiling:** record when and why threads stop running (waiting on locks, I/O, page faults) – the time CPU profiles cannot see.
3. **Lock-contention profiling:** count how often and how long threads wait on each lock, identifying the “hot” locks to remove.
4. **Allocation profiling:** find code paths that allocate most often.
5. **Tail-latency tracing:** for requests above the p99 threshold, record stage timings and context to find what the slow ones have in common (large bodies, many headers, specific rules, slow clients).
6. **Hardware counters:** measure cache misses and branch mispredictions to guide data layout changes.

##### Minute details that matter:

- Profiling runs in a benchmark lab under realistic replayed traffic and, with low-overhead sampling, on a small slice of production.
- Coordinated-omission-aware load generators (e.g., wrk2) are used so p99 numbers are not artificially flattering.

##### Contribution to the results

This is how every bottleneck behind the 30–40% and 25% results was found and verified.

#### 4.4.5 Benchmark lab and CI performance gates

##### In simple terms

An automatic speed test that runs on every code change, so improvements are proven and regressions are caught before release.

**What it is:** Micro-benchmarks for each component, end-to-end load tests on dedicated hardware, and continuous-integration checks with thresholds for throughput and p99.

**Why it is needed:** Performance gains are easily lost by later changes. Automated gates lock them in.

**How it works, step by step:**

1. Micro-benchmarks (e.g., Criterion for Rust, Google Benchmark for C++) measure parse/normalize/match speed on standard request sets.
2. End-to-end tests replay realistic traffic mixes (sizes, headers, attack share) at fixed request rates.
3. Results are compared with the main branch; a regression beyond a threshold (e.g., 2–3%) fails the build.
4. Trends are charted so slow drifts are noticed.

## 5. Workflows Explained End to End

This section follows the system through real situations step by step. Step numbers in 5.1 match the numbered circles in Figure 2.

### 5.1 Life of one request through the proxy

#### Example

A mobile app sends **POST /api/v1/orders** with a small JSON body to a service protected by the proxy.

| Step | What happens technically                                                                                                                                                                          | In simple terms                                                                              |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| 1    | The app's connection arrives at the server.                                                                                                                                                       | An order arrives at the kitchen door.                                                        |
| 2    | The network card places packets on core 5's queue; the kernel queues the connection on worker 5's SO_REUSEPORT socket; epoll signals worker 5.                                                    | The doorman sends it straight to cook #5, no queue at a single entrance.                     |
| 3    | Worker 5's event loop wakes and runs the accept handler; the socket is set non-blocking and registered for reads.                                                                                 | Cook #5 notes "new order at table 5" and carries on with other work until it is ready.       |
| 4    | A connection slot is taken from worker 5's slab; idle and header timers are set in the timer wheel.                                                                                               | A ticket is clipped to the order board.                                                      |
| 5    | When data is ready, bytes are read directly into a pooled 16 KB buffer.                                                                                                                           | The order is written once on one ticket.                                                     |
| 6    | The Rust parser scans with SIMD, records slices for method, path, headers; body framing is validated. Headers complete.                                                                           | The ticket is read and its parts identified – no rewriting.                                  |
| 7    | The C++ normalizer lower-cases names, decodes the path in place, maps known headers to IDs.                                                                                                       | Messy handwriting is tidied into a standard form.                                            |
| 8    | The Rust matcher scans path, query, headers and body against the compiled rule database; no rules hit.                                                                                            | The order is checked against the "forbidden items" list in one glance.                       |
| 9    | The policy bytecode runs: route /api/v1/* → validate score → ALLOW; add request-ID header.                                                                                                        | The head cook approves it.                                                                   |
| 10   | An idle keep-alive HTTP/2 connection to the orders cluster is taken from worker 5's pool; the request is sent as a new stream.                                                                    | An already-open line to the supplier is used.                                                |
| 11   | writev sends [request line][new header][original headers][body] in one call.                                                                                                                      | The order is passed on without copying it out again.                                         |
| 12   | The backend replies; the same worker streams the response back to the app through the same path, buffers are returned to the pool, the request arena is released, and stage timings are recorded. | The dish comes back, the station is cleaned in one sweep, and the stopwatch times are noted. |

**Key point:** from arrival to departure, the request is handled by **one worker on one core**, with **no locks**, **no copies** of the request bytes, and **no waiting** on the network. That is exactly why throughput went up and tail latency went down.

## 5.2 A malicious request is blocked early

1. An attacker sends GET /search?q=%2527%2520OR%25201%3D1 (double-encoded SQL injection).
2. Steps 1–6 happen as usual; the parser accepts the request as valid HTTP.
3. The **normalizer** decodes the query twice safely, producing q=' OR 1=1 in the normalized view, while keeping the original for logs.
4. The **rule matcher** scans the query field; the SQL-injection pattern group reports a hit; confirmation passes. The rule is marked terminal, so **early exit** skips scanning remaining headers and body.
5. The **policy executor** returns BLOCK with the rule ID as the reason.
6. The **reply builder** sends a pre-built 403 response from memory – no formatting work needed.
7. No upstream connection is used; buffers and arena are released immediately. Total time is lower than an allowed request, because work stopped early.

## 5.3 A slow client sends a request in tiny pieces

1. A client (on a poor network, or an attacker) sends a request a few bytes at a time.
2. Each time bytes arrive, epoll wakes the worker; the Rust parser resumes exactly from its saved position – it never re-scans earlier bytes.
3. Between pieces, the worker serves thousands of other connections – the slow client costs almost nothing while it waits.
4. The **header timer** in the timer wheel limits how long headers may take. If the client is too slow, the connection is closed and its slot and buffers are reclaimed.
5. Because of per-connection memory caps, the slow client cannot hold more than its small allowance, no matter how long it takes.

## 5.4 Large upload with a slow backend (backpressure)

1. A client uploads a 200 MB file to /upload; the backend can only accept data slowly.
2. Body chunks are read into pooled buffers and streamed through inspection (with a per-request inspection limit on how much body is scanned).
3. The upstream connection's send buffer fills; the connection's buffered-byte count passes the **high watermark**.
4. The worker **stops reading** from the client socket – TCP flow control then naturally slows the client.
5. As the backend drains data and buffered bytes fall below the **low watermark**, reading from the client resumes.
6. Memory for this upload never exceeds its cap, however big the file or slow the backend, so other requests on the worker are unaffected.

## 5.5 Rule or configuration update without pausing traffic

1. A new rule set arrives at the main thread from the control plane.
2. The main thread validates it and compiles the new pattern database and policy bytecode in the background (workers are unaffected).
3. Compilation includes performance checks; a rule that could be slow on crafted input is rejected.
4. The main thread atomically swaps the shared pointer to the new version (RCU).
5. Each worker sees the new version at its next request boundary; in-flight requests finish on the old version.
6. When no worker references the old version, it is freed. Traffic never pauses and no lock is taken on the request path.

## 5.6 Traffic spike and overload protection

1. Traffic suddenly doubles (a flash sale or an L7 flood).
2. Per-worker stats show event-loop lag rising; the overload manager sees memory and queue depth climbing.
3. Level 1 actions: shorter idle timeouts, fewer kept-alive idle connections – freeing memory.
4. If pressure continues, level 2: fast 503 for low-priority routes and refusing some new connections, which costs almost no CPU.
5. Because all workers are isolated and memory is bounded, admitted requests keep their normal latency rather than all requests slowing down.
6. As traffic falls, actions are reversed step by step.

## 5.7 The performance-engineering workflow (how the gains were achieved)

Figure 3 shows the loop. Here is each step in detail.

1. **Measure the baseline.** Replay a realistic traffic mix (request sizes, header counts, attack share, HTTP/1.1 vs HTTP/2) on dedicated servers at fixed rates. Record throughput at a target CPU level, and p50/p99/p999 inspection latency per stage.
2. **Profile.** Under that load, capture CPU flame graphs, off-CPU (waiting) profiles, lock-contention reports, allocation profiles, hardware counters, and traces of the slowest 1% of requests.
3. **Find the cause.** Rank bottlenecks by cost. For each, ask “what is this code doing that it does not need to do?” – copying, allocating, locking, re-scanning, branching unpredictably, or waiting.
4. **Fix.** Make a focused change in C++ or Rust: e.g., replace a copy with a slice, a lock with per-worker data, a regex loop with a compiled matcher.
5. **Verify.** Run micro-benchmarks and the full A/B load test on identical hardware; check that correctness tests, fuzzing and security test suites still pass. Add or tighten the CI performance gate.
6. **Roll out.** Deploy to a canary slice of servers; compare live p99 and CPU with the control group; expand gradually; watch dashboards.

7. **Repeat** with the next biggest bottleneck until remaining gains are too small to be worth the complexity.

### Where the improvements came from (illustrative breakdown)

| Bottleneck found by profiling                                               | Tool that revealed it                         | Fix applied                                                            | Main effect                                      |
|-----------------------------------------------------------------------------|-----------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------|
| Per-rule regex evaluation dominated inspection CPU and caused slow outliers | CPU flame graph + tail-latency traces         | Compiled multi-pattern matcher (Rust) with early exit and time budgets | Largest p99 drop; big CPU saving                 |
| Request bytes copied 4–6 times through the pipeline                         | Flame graph (memcpy hot), cache-miss counters | Zero-copy slices, in-place normalization, scatter-gather writes        | Throughput gain; fewer cache misses              |
| Global locks on config reads, stats and counters                            | Lock-contention and off-CPU profiles          | RCU config, per-worker stats and state                                 | Throughput scales with cores; p99 spikes removed |
| Thousands of small allocations per request                                  | Allocation profile                            | Per-request arenas, slab-allocated connection state, buffer pools      | Lower CPU and allocator stalls                   |
| New upstream connections under load                                         | Off-CPU profile + upstream timing             | Per-worker keep-alive and HTTP/2 connection pools                      | Fewer handshakes and latency spikes              |
| Re-parsing when requests arrived in pieces                                  | Tail-latency traces of slow clients           | Resumable streaming Rust parser                                        | Tail latency with slow clients reduced           |
| Generic policy interpreter with heavy branching                             | Branch-miss counters                          | Policy compiled to bytecode / decision tree                            | Faster, more predictable policy step             |
| Memory spikes under slow backends and big bodies                            | Memory and latency correlation                | Bounded buffers, watermarks, backpressure                              | Stable p99 under stress                          |

Together these changes produced the **30–40% throughput increase** (range depends on traffic mix – header-heavy and HTTP/1.1 traffic benefited most) and the **25% p99 inspection-latency reduction**.

## 5.8 Safely shipping new data-plane code

1. **Correctness first:** unit tests, differential tests (new parser vs old parser on millions of real requests must agree), and continuous fuzzing of the parser and normalizer.
2. **Security review:** request-smuggling and normalization test suites; OWASP attack corpora through the rule matcher.
3. **Shadow mode:** new components run alongside the old ones on live traffic, compare outputs, and log differences – but the old path still makes the decision.
4. **Canary:** a small set of servers switches to the new path; throughput, p99, error rates and block rates are compared with control servers.
5. **Gradual rollout** across regions, with a runtime feature flag for instant fallback to the previous implementation.

6. **Post-rollout monitoring:** dashboards and alerts on per-stage p99 and CPU per request to catch regressions early.

## 6. Key Design Decisions and Trade-offs

| Decision               | Choice made                                                     | Why                                                                                                | Trade-off accepted                                                                   |
|------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Language per component | Rust for parser, matcher, policy; C++ for core-integrated parts | Rust's memory safety where untrusted input is handled; C++ where it meets existing proxy internals | Two toolchains and an FFI layer to maintain                                          |
| Concurrency model      | Event loop per pinned worker, shared-nothing                    | No locks, cache-warm data, linear scaling                                                          | Harder to balance very uneven long-lived connections                                 |
| Memory model           | Pools, slabs, per-request arenas, hard caps                     | Predictable, fast, and safe under attack                                                           | Careful sizing; very large requests need special handling                            |
| Data movement          | Zero-copy slices and in-place edits                             | Least CPU and cache pressure                                                                       | Lifetimes of buffers must be tracked carefully (reference counts, Rust borrow rules) |
| Rule evaluation        | Pre-compiled multi-pattern database + early exit                | Cost barely grows with rule count; stable tail                                                     | Compile step on every rule update; more complex tooling                              |
| Config updates         | RCU pointer swap                                                | No locks on the hot path                                                                           | Old and new config briefly in memory together                                        |
| Upstream connections   | Per-worker pools                                                | No locks; warm connections                                                                         | More total connections to backends than a shared pool                                |
| Overload behaviour     | Shed early and cheaply                                          | Protects latency for admitted requests                                                             | Some requests rejected during extreme spikes                                         |

## 7. Recap: Explaining the Project in Two Minutes

---

### A short spoken version

“Every request to our security platform passes through an Envoy/Nginx-style proxy, so its inline inspection path decides both our cost and our latency. I rebuilt the core inline components – the HTTP parser, header normalizer, rule matcher, connection-state manager and policy executor – in Rust and C++. Rust went where we handle untrusted bytes, C++ where we integrate with the proxy core, with a zero-copy bridge between them.

Each worker thread is pinned to a core and shares nothing: its own event loop with non-blocking I/O, its own buffers, arenas and upstream connection pools. Requests are read once into pooled buffers and every stage works on slices of that memory. Rules are compiled into one multi-pattern database with early exit, policies into bytecode, and config is swapped with RCU so the hot path takes no locks. Memory is bounded everywhere with watermarks and backpressure.

I drove it with profiling – CPU flame graphs, off-CPU and lock-contention analysis, and traces of the slowest 1% – and locked in gains with CI performance gates. The result was 30–40% more throughput per server and a 25% lower p99 inspection latency.”

### Key takeaways

- **Never wait** – async, non-blocking I/O keeps every core busy.
- **Never share** – worker isolation removes locks and cache bouncing.
- **Never copy** – slices over one buffer instead of new strings.
- **Never grow without limit** – bounded memory keeps the tail stable.
- **Reuse everything** – buffers, connection slots, upstream connections.
- **Measure, don't guess** – profiling found each bottleneck; benchmarks proved each fix.